

Teamder — Audit + Migration Planning

מעבר מה-ruleset שבפרודקשן למודל החדש · 25.09.2026 · ניתוח בלבד — לא בוצע שום שינוי, commit או deploy

ענף / HEAD
Ruleset בפרודקשן
Ruleset מיועד
שיטה

ריק בתחילת הסבב ובסופו
git status · 4fbf441 · perf/firestore-read-costs
עודכן 22.09.2026 (הגלגול לאחר שאחרי תקרית ה-is list
867bea46-48ed-45c9-9a3b-57d693443bbf
(
firestore.rules ב-HEAD (טרם נפרס)
קריאת קוד + הרצת emulator מקומי על demo-soccer. אפס קריאות לפרודקשן בסבב הזה.

התשובה הקצרה לשאלה המרכזית (§21)

כן — R-B-2 חמור דיו להצדיק שינוי rules לפני שהריפקטור והגרסה מסתיימים, אבל

לא את ה-ruleset החדש כולו

. ההוכחה שנוספה בסבב הזה משנה את הסיווג: allow read: if isSignedIn() על /users אינו רק קריאת מסמך לפי uid

ידוע — הוא מתיר גם

list ו-query על כל האוסף

. כל מי שמתקין את האפליקציה ומקבל session אנונימי (שנייה אחת, בלי אימות) יכול להוריד את בסיס המשתמשים כולו: שם, אימייל,

טוקני push, קואורדינטות אזור הבית, זמינות, ושיוך מזמין. זה חוסר הגנה על נתונים שקיים

בלי קשר

לריפקטור, והוא לא צריך לחכות לו.

1 Deployed מול diff — Intended מונטי

ההשוואה נעשתה על נתיבי match מנורמלים ומסודרים, לא על טקסט. הטקסט שונה במאות שורות (הערות בלבד ברוב המקרים) והוא חסר משמעות כאן.

מדד	בפרודקשן	ב-HEAD
נתיבי match ייחודיים	60	61
אוספים שקיימים רק בצד אחד	— (אין)	match /usersPublic/{uid}
catch-all / deny מפורש	אין — ברירת המחדל היא deny	זהה

המסקנה המבנית

שום אוסף, שום allow ושום יכולת שקיימת היום בפרודקשן לא נעלמה ב-HEAD בגלל מחיקה. כל ההפרש הפונקציונלי מרוכז בשני מקומות: אוסף חדש אחד (/usersPublic), ועשר נקודות שבהן isSignedIn() הוחלף ב- isFullAccount(). אין ב-HEAD הפתעות מחוץ לשתי הקבוצות האלה.

2 מה חשוף ב-/users — לפי קטגוריה

מסמך המשתמש מכריז על 35 שדות ברמה הראשונה, ועוד שלושה אובייקטים מקוננים. להלן קטגוריות ושמות שדות בלבד — לא הודגם, נקרא או צוטט שום ערך אמיתי.

קטגוריה	שדות	רגישות
מזהה ישיר	email	גבוהה

קטגוריה	שדות	רגישות
מיקום	availability.homeCityLng , availability.homeCityLat , availability.homeCity רדיוס	גבוהה
ערוץ התקפה	fcmTokens	גבוהה
גרף חברתי ושייך	invitedAt , invitedByTargetId , invitedByType , invitedBy , friends , personalGroupId	בינונית
שייוך / מקור	at , linkId , gameId , campaign , source , acquisition	בינונית
התנהגות ונוכחות	discipline , platform , lastBroadcastAt , lastSeenAt	בינונית
העדפות	dmFriendsOnly , newGameSubscriptions , notificationPrefs (18 שדות) , availability (13 שדות)	נמוכה
סטטיסטיקה ומשחק	jersey , position , achievements , stats (15 שדות)	נמוכה
ציבורי בפועל	photoUrl , avatarId , name , id	אפס
תשתית	qa , isGuest , onboardingCompleted , updatedAt , createdAt	אפס

ארבעת השדות בשורה "ציבורי בפועל" הם בדיוק מה שהמראה מפרסמת. כל השאר — 31 שדות ושלושה אובייקטים — נגיש היום לכל session מאומת, אנונימי כולל.

3

עשר נקודות ה־ isFullAccount() — מה הן מחליפות

ב־HEAD	בפרודקשן היום
— /users/{uid}	allow read: if isSignedIn()
allow get: isSelf(uid) isFullAccount()	
— /users/{uid}	אותו read יחיד
allow list: isFullAccount()	
— /users/{uid}	allow create: if isSelf(uid) && ...
allow create: isFullAccount() && isSelf(uid) && ...	
— /usersPublic/{uid}	האוסף כולו לא קיים ב־rules
allow list: isFullAccount()	
— /rounds/{id}	allow read: if isSignedIn()
allow read: isFullAccount()	
— /playerStats/{uid}	allow read: if isSignedIn()
allow read: isFullAccount()	
— /ratings/{ratedUid}	allow read: if isSignedIn()
allow read: isFullAccount()	

`allow read: if isSignedIn()``/users/{uid}/seasonTitles/{titleId}``allow read: isFullAccount() —``allow read: if isSignedIn()``— /pairStats/{pair}``allow read: isFullAccount()`

השאלה: האם `allow read: if isSignedIn()` על `/users/{uid}` מתיר גם קריאת מסמך לפי `uid` ידוע וגם `query` על האוסף? ההוכחה הורצה על שני ה-rulesets ב-emulator מקומי מול `demo-soccer` — אותה תשתית שמריצה את 268 בדיקות ה-rules. אפס קריאות פרודקשן.

הפעולה	דורש	פרודקשן	HEAD
<code>getDoc /users/{uid}</code> (uid ידוע)	אנונימי	ALLOW	DENY
<code>getDocs /users</code> — האוסף כולו	אנונימי	ALLOW	DENY
<code>where('invitedBy','==',uid)</code>	אנונימי	ALLOW	DENY
<code>where('email','==', ...)</code>	אנונימי	ALLOW	DENY
<code>getDoc /users/{self}</code> — המסמך של עצמי	אנונימי	ALLOW	ALLOW
<code>updateDoc /users/{self}</code>	אנונימי	ALLOW	ALLOW
<code>setDoc /users/{self}</code> — יצירה	אנונימי	ALLOW	DENY
כל ארבע הראשונות	חשבון מלא	ALLOW	ALLOW
כל ארבע הראשונות	לא מחובר	DENY	DENY

התשובה

`get = allow read`

וגם

`list`. החשיפה בפרודקשן איננה "מי שידוע uid יכול לקרוא מסמך" — היא אינדקס מוריד של כל בסיס המשתמשים, כולל `query` לפי אימייל וכולל `query` לפי מזמין. זו העדות המרכזית של הסבב, והיא מעלה את R-B-2 מדרגה.

שני דברים שההוכחה הבהירה על המודל החדש: קריאת המסמך של עצמי נשמרת דרך `isSelf(uid)`, ולכן אורח-חדש שקורא את עצמו לא נפגע; ואילו `create` נחסם לאנונימי. זה תואם את הקוד בכוונה מלאה — `userService.signInAsGuest` בונה `User` שקיים ב-runtime בלבד ואינו כותב מסמך כלל (`buildGuestUser`, ההערה "No /users doc is created"). הסתירה היחידה שיכולה הייתה לצמוח כאן — אורח שכותב `isGuest:true` ל-Firebase — לא קיימת בקוד.

שאלת ההשקה של החשבון (הבדיקה שהייתה יכולה להפיל את המודל)

`authUpgrade` מקדם אורח באמצעות `linkWithCredential`, כלומר אותו `uid` נשאר. אם ה-token ID היה ממשיך לדווח `sign_in_provider: 'anonymous'` אחרי הקישור, `isFullAccount()` היה מחזיר `false` למי שבדיוק נרשם — ו-`allow create` על `/users` היה חוסם את כתיבת מסמך המשתמש שלו. זו הייתה שוברת-הרשמה. נמדד ב-Auth emulator:

1. session טרי	אנונימי	<code>isAnonymous=true</code>	<code>sign_in_provider=anonymous</code>	<code>providerData=[]</code>
2. link מיד אחרי session אותו		<code>isAnonymous=false</code>	<code>sign_in_provider=password</code>	<code>providerData=[password]</code>
3. אחרי התחברות סיסמה חדשה		<code>isAnonymous=false</code>	<code>sign_in_provider=password</code>	<code>providerData=[password]</code>

הטוקן נושא את ה-provider החדש גם ללא רענון מאולץ (נמדד פעמיים, עם `getIdTokenResult(true)` ובלעדיו) — ה-SDK מחליף את הטוקן בזה שהוחזר מקריאת ה-link. אין חלון של טוקן מיושן, ולכן `isFullAccount()` מחזיר `true` מיד. **הסתייגות** נמדד על ה-emulator, לא על Identity Platform בפרודקשן. זו הנקודה היחידה במודל שתלויה בהתנהגות של Firebase ולא בקוד שלנו, והיא ראויה לאימות אחד על מכשיר אמיתי לפני ה-deploy.

5 Old binary dependency matrix

עובדה שמשנה את המסגור: מצב אורח לא נולד בריפקטור הזה. `signInAsGuest` נכנס ב- `c1c9f7c` בתאריך **17.06.2026** (Apple) — `review fixes 5.1.1(v): browsing must not require registration` — כלומר בינאריים מותקנים מאז יוני יוצרים sessions אנונימיים. זה המקור לחשבונות האנונימיים הקיימים, וזה מה שהופך את שאלת ה-old binary לאמינית ולא תיאורטית.

המראה (`hydratePublicUsers`) נכנסה ב- `ffcba25` בתאריך **24.09.2026** ועברו אליה שני מקומות בלבד: הידרציית הרוסטר ב- `gameStore` וכרטיס השחקן. כל שאר קוראי `/users` נשארו על המסמך הקנוני.

משטח	מקור בבינארי הישן	סוג קריאה	מה הוא צריך	מה נשבר אם נדחה
קריאת עצמי — לא נפגע				
בוט האפליקציה, פרופיל, הגדרות	<code>getCurrentUser</code> → <code>docs.user(self)</code>	<code>get</code>	המסמך של עצמו	כלום. <code>isSelf(uid)</code> נשמר, הוכח בטבלה של §4.
אורח בבינארי ישן	<code>getCurrentUser</code> ענף <code>fbUser.isAnonymous</code>	—	אפס קריאות	כלום. אורח לא קורא ולא כותב מסמך משתמש בשום גרסה.
קריאת אחרים, session אנונימי — נפגע (נמדד ב-emulator)				
רוסטר משחק ציבורי	<code>gameStore.hydratePlayers</code> → <code>hydrateUsers</code>	<code>list</code> <code>documentId()</code> <code>in</code>	שם + אווטאר	כל השמות ברוסטר יורדים ל-"...". <code>try/catch</code> קיים בבינארי הישן, ולכן אין קריסה — רק דממה.
כרטיס שחקן	<code>getUserById</code>	<code>get</code>	המסמך של האחר	המסך מציג <code>he.playerCardNotFound</code> — "שחקן לא נמצא". זו הרגרסיה הנראית הגרועה ביותר: הודעה שגויה, לא מסך ריק.
כרטיס שחקן — סטטיסטיקה	<code>/playerStats/{uid}</code>	<code>get</code>	רולאפ	בלוק הסטטיסטיקה ריק (<code>.catch</code> קיים).
כרטיס שחקן — דירוג	<code>/ratings/{uid}</code>	<code>get</code>	דירוג פנימי	לא מוצג.
כימיה / ראש-בראש	<code>/pairStats/{pair}</code>	<code>get</code>	זוגות	הסקשן לא נטען.
היסטוריית מחזורים	<code>/rounds/{id}</code>	<code>get</code>	מחזור	ריק.
תארי עונה	<code>/users/{uid}/seasonTitles</code>	<code>get</code>	תארים	ריק.
קריאת אחרים, חשבון מלא — לא נפגע				

משטח	מקור בבינארי הישן	סוג קריאה	מה הוא צריך	מה נשבר אם נדחה
שחקני מועדון, אישורי אדמין, פרטי מועדון, כימיה	groupService.hydrateUsers	list	מסמכי משתמש מלאים	כלום. allow list: isFullAccount() עובר, הוכח. ובנוסף כל המשטחים האלה כבר חסומים ב־ /groups על חברות, שזהה בשני ה־rulesets.

גילוי — לא נפגע בשום מצב				
פיד משחקים ציבוריים	where('visibility','','=','public')	list	משחקים	ALLOW בשני ה־rulesets, אנונימי כולל.
פיד מועדונים ציבוריים	/groupsPublic	list	מועדונים	ALLOW בשני ה־rulesets.

הסיכום הכמותי של ה־old binary
ההידרדרות של אורח בבינארי ישן היא **שקטה וחלקית, לא קריסה**
: שמות רוסטר ל-"...", וכרטיס שחקן שאומר "שחקן לא נמצא". חשבון מלא בבינארי ישן **אינו מאבד דבר**
— זו התובנה שמקטינה את הסיכון של ה־deploy בסדר גודל, כי רוב האוכלוסייה היא חשבונות מלאים.

6 מה אנחנו באמת יודעים על אימוץ גרסאות

אין לנו טלמטריית אימוץ גרסאות אמינה.
זו התשובה הישרה, ולא שווה לבנות עליה החלטה שמתחזה למדידה.

מה שכן קיים, ומה שכל מקור באמת מכסה:

מקור	נושא גרסה?	הטיה / מה הוא לא מכסה
touchPresence → /users	לא	כותב platform ו־lastSeenAt בלבד. אין שדה גרסה על מסמך המשתמש. זה הפער המרכזי.
describeThisDevice()	בונה appVersion	פונקציה מתה — מוגדרת ב־notificationsService ואינה נקראת. ה־appVersion שהיא בונה לא נשמר לשום מקום.
אוסף errors	כן	מכסה רק מי שנתקל בשגיאה. הטיה חמורה — לא מדד אוכלוסייה.
דוחות feedback	כן	רק מי שטרח לשלוח. הטיה חמורה.
Joryio — appVersion כתכונת	כן	נקבע ב־identify, וב־bindIdentity אורח אנונימי אינו מזוהה בכוונה (if (subject.isAnonymous) return 'none'). כלומר דווקא האוכלוסייה שבסיכון — sessions אנונימיים על בינארי ישן — היא זו שאין לה תכונת גרסה ב־Joryio.
Play Console / App Store Connect	כן, מהימן	המקור הסמכותי היחיד למכשירים פעילים לפי גרסה. חיצוני, ולא נשאל בסבב הזה.

מקור	נושא גרסה?	הטיה / מה הוא לא מכסה
appConfig.latestVersion / minimumSupportedVersion	—	אלה ידידות שליטה, לא מדידה. הן אומרות מה אנחנו דורשים, לא מה מותקן.

לכן: המשפט "416 חשבונות אנונימיים" אינו "416 בינאריים ישנים פעילים". מספר חשבונות ב־Auth הוא מספר **מצטבר של יצירות** — כל התקנה מחדש, כל ניקוי נתונים וכל session של רובוט Play יוצרים אחד. הוא אינו מכיל מידע על גרסת הבינארי ואינו מידה לתפוצה. ההחלטה לפרוס rules חייבת להישען על Play/ASC, לא על הספירה הזו.

7 תאימות /usersPublic — האם הלקוח החדש צריך list

כן, לחשבון מלא בלבד — ובמדויק בגלל תמחור קריאות. hydratePublicUsers מפצל בכוונה לשני מסלולים לפי : auth.currentUser?.isAnonymous

הצופה	מה הלקוח עושה	סוג קריאה	למה
אורח	uid לכל getDoc	get	אורח מסתכל על משחק אחד. שני תריסר get, לא אינדקס.
חשבון מלא	where(documentId(), 'in', chunk) ב-30	list	רוסטר של 200 חברים ב-7 queries במקום 200 קריאות מחויבות.
הזמנה אישית	hydratePublicUsers([invitedBy])	get	uid אחד — אורח לא נזקק ל-list אפילו כאן.

ה-rules המיועדים מתלבשים על הפיצול הזה אחד לאחד, והוכח:

פעולה על /usersPublic	אורח	חשבון מלא	לא מחובר
uid לפי get	ALLOW	ALLOW	ALLOW
list — האוסף כולו	DENY	ALLOW	DENY
list — documentId() in [...]	DENY	ALLOW	DENY
כתיבה כלשהי	DENY	DENY	DENY

ה- get: if true הפתוח גם למי שאינו מחובר אינו רשלנות — הוא מה שמזין את דפי ה-web הציבוריים (עמוד משחק, עמוד הזמנה אישית), והוא מחזיק ארבעה שדות שאותם עמודים כבר מציגים.

המצב בפרודקשן היום, שראוי להיאמר במפורש

ל- /usersPublic אין match ב-ruleset שבפרודקשן, ואין catch-all — ולכן

ברירת המחדל דוחה את כולם, כולל חשבונות מלאים

. הוכח למעלה: get ו-list נדחים לשלושת סוגי הצופים. המראה קיימת בנתונים, ה-trigger מתחזק אותה, ואף אחד לא יכול לקרוא אותה. כל משטח שנבנה עליה מרנדר ריק בשקט.

8 שלמות המראה — קריאה בלבד

scripts/auditUsersPublic.mjs הוא הכלי לכך, והוא read-only בהצהרה ובפועל. הוא בודק שש שאלות, בסדר החשיבות שלהן: missing (משתמש ללא מראה — אלה שנשברים), orphan (מראה שהמשתמש שלה נמחק), extraFields (מראה שנושאת משהו מחוץ ל-allowlist — זו הבדיקה שהייתה הופכת לממצא פרטיות), idMismatch, blankName, avatarMismatch.

לא הורץ בסבב הזה, במקוון: הוא סורק את שני האוספים בעמודים של 300 מסמכים, וזו בדיוק ההגדרה של mass read על פרודקשן שהסבב אוסר. הידע שיש לנו הוא מהרצה קודמת (24.09): 708 מראות מולאו ואומתו, ובתוכן עשרה stubs עם שם ריק. לא אומת מחדש

נקודה שמשנה את סדר הפעולות

ההערה בראש הסקריפט מנסחת את התלות נכון: "ה-rules הם מה שהופך מראה חסרה לנראית למשתמש". כל עוד /users פתוח, מראה חסרה נסתרת — הלקוח החדש פשוט ייפול חזרה למסמך הקנוני. ברגע שנסגור, מראה חסרה הופכת לשם ריק בכל משטח ציבורי, ושום דבר לא ירשום את זה.

9 אבטחת המראה — מה לא חוצה

ה־`allowlist` בשרת: `const USERS_PUBLIC_FIELDS = ['name', 'avatarId', 'photoUrl']`, והמסמך שנכתב הוא `{id, name, avatarId?, photoUrl?}`. שתי החלטות מימוש מחזיקות את זה:

- `set(..., {merge: false})` — המראה **נבנית מחדש** בכל כתיבה. עם `merge`, שדה שפעם היה ב־`allowlist` היה שורד לנצח אחרי שהוא ממנה.
- שדות ריקים **מושמטים** ולא נכתבים כ־`null`, כדי שהמסמך ישמור על הצורה שה־`converter` בלקוח מצפה לה.
- מחיקה מוחקת: משתמש שנמחק לא משאיר שם ציבורי מאחוריו.

הקטגוריה שנשאלה	חוצה למראה?	הנימוק
אימייל / טלפון	לא	אינם ב־ <code>allowlist</code> . אין שדה טלפון על המסמך מכל מקום.
טוקני <code>push</code>	לא	אינם ב־ <code>allowlist</code> .
זמינות וקואורדינטות בית	לא	אינם ב־ <code>allowlist</code> .
שיוך מזמין	לא	אינו ב־ <code>allowlist</code> .
סטטיסטיקה	לא	אינה ב־ <code>allowlist</code> .
תפקידים / הרשאות	לא	אינם ב־ <code>allowlist</code> , ואינם קיימים על מסמך המשתמש כלל.
שם תצוגה	כן, בכוונה	ראו הממצא מתחת.

ממצא נמוך — `name` הוא טקסט חופשי בלי אימות בשרת

ה־`rules` אינם מאמתים את `name` בכתיבה ל־`/users`; `isEmailLikeName` ו־`isReservedName` נאכפים בלקוח בלבד.

(`userService`). מי ששם אימייל או טלפון בשם התצוגה שלו מפרסם אותו `world-readable` דרך המראה. הפגיעה מוגבלת לעצמו, ובינארי שעבר שינוי יכול לעקוף את הבדיקה. שווה לשקול העברת אותה בדקה ל־`rules` או ל־`trigger` בעתיד — לא כחלק מהמיגרציה הזו.

10 מודל הכתיבה ל־`/usersPublic`

שלוש שכבות, כל אחת מספיקה בעצמה:

- Rules** — `allow write: if false`. אין חריג, גם לא לבעלים. ההערה מנסחת את הסיבה נכון: לקוח שיכול לכתוב כאן יכול לפרסם כל שם תחת כל `uid`, ובעלים שכותב את המראה של עצמו הוא בדיוק איך ששני המסמכים מתפצלים.
- Converter בלקוח** — `publicUserConverter.toFirestore()` **זורק** `'usersPublic is server-written only'`. כתיבה מקרית נכשלת בזמן פיתוח, לא ב־`rules` בפרודקשן.
- טיפוס** — `PublicUser` הוא `interface` נפרד עם ארבעה שדות, ולא `Partial<User>`. קורא שמושיט יד לשדה חמישי מקבל שגיאת קומפילציה, לא `undefined`.

ה־`trigger` `syncUserPublic` הוא `onDocumentWritten('users/{uid}')` עם שומר־סף: אם אף אחד משלושת שדות ה־`allowlist` לא השתנה — יציאה מיידית. זה נחוץ ולא אופטימיזציה, כי מסמך משתמש נכתב בכל עריכת זמינות, כל עדכון סטטיסטיקה, כל רענון טוקן וכל `ping`.

נוכחות. הקורא בלקוח (`fromFirestore`) בונה שדה־שדה — כלומר שדה שיתווסף למראה **לא יעשה דבר בקריאה** עד שיתווסף גם לקורא. עבור projection ציבורית זו ההתנהגות הרצויה.

11 אפשרות A — לפרוס את ה-ruleset החדש כולו, עכשיו

מה זה	deploy של firestore.rules מ-HEAD לפרודקשן לפני שהלקוח החדש בחנויות.
מה זה סוגר	הכול: הדליפה של /users , האוסף החסר, ותשע נקודות ה- isFullAccount() .
מה זה שובר	מה שהוכח ב-5: אורחים על בינאריים מיוני ואילך מאבדים שמות רוסטר ומקבלים "שחקן לא נמצא" בכרטיס שחקן. חשבונות מלאים — כלום.
הסיכון	בינוני ההידרדרות שקטה, אבל היא פוגעת דווקא בחוויית האורח — המשטח שכל הריפקטור נבנה כדי לשפר. ואם המראה חסרה למישהו, השם שלו ייעלם בלי שאיש ידע.
תלוי ב-	הרצת auditUsersPublic נקייה, ואימות ה-provider אחרי link על מכשיר אמיתי.

12 אפשרות B — רק בלוק /usersPublic , כשינוי מוסף

מה זה	הוספת match /usersPublic/{uid} בלבד. /users ותשע הנקודות האחרות נשארות כפי שהן בפרודקשן.
מה זה סוגר	את הדממה: המראה הופכת לקריאה, ומשטחים ציבוריים מתחילים לרנדר. לא סוגר שום דבר אבטחתי.
מה זה שובר	שום דבר, בהוכחה. האוסף היום הוא default-deny לכולם, ולכן אין לקוח בשום גרסה שמסתמך על ההתנהגות הנוכחית שלו. כל allow שנוסיף הוא הרחבה טהורה.
הסיכון	אפס תפעולי הסיכון היחיד הוא סמנטי: get: if true חושף ארבעה שדות למי שאינו מחובר. זו החלטת מוצר שכבר נלקחה במפורש למען דפי ה-web.

13 אפשרות B — C פלוס סגירת /users בלבד

מה זה	הבלוק של /usersPublic , ובנוסף שלוש שורות על /users : get ל-self או חשבון מלא, list לחשבון מלא, create לחשבון מלא. שאר שמונה הנקודות (rounds , playerStats , ratings , pairStats , seasonTitles) נשארות כפי שהן.
מה זה סוגר	את כל הדליפה שנושאת מזהים אישיים: אימייל, טוקנים, קואורדינטות בית, שיוך, העדפות. זה כל ה-PII.
מה זה שובר	רק שתי השורות הראשונות בטבלת ה-old binary: שמות רוסטר ל-"...", וכרטיס שחקן שאומר "שחקן לא נמצא" — לאורחים על בינארי ישן בלבד. סטטיסטיקה ודירוגים ממשיכים לעבוד להם.
הסיכון	נמוך שטח הפגיעה קטן ומוגדר, והתועלת האבטחתית היא כמעט כל התועלת של A.

14 אפשרות D — לחכות, ולפרוס יחד עם הגרסה

מה זה	לסיים את הריפקטור, לשלוח את הבינארי לשתי החנויות, לתת לאימוץ להתקדם, ואז לפרוס את ה-ruleset כולו.
מה זה סוגר	הכול — בסוף.
מה זה שובר	שום דבר בלקוח.

זה בדיוק הסיכון שקל לפספס: "לחכות" נשמע כמו האפשרות השמרנית, אבל היא משאירה את בסיס המשתמשים כולו פתוח להורדה בכל יום נוסף של המתנה — זמן ההמתנה כאן נמדד בשבועות, כי הוא כולל שתי סבבי ביקורת חנות ואימוץ. נוסף על כך, ההיסטוריה של הפרויקט אומרת שהמתנה הזו לא מסתיימת: gates של `appConfig` נשארו תקועים כמעט שלושה חודשים, וה-`popup` הרך היה מת שבע גרסאות.

15 סמנטיקת `isFullAccount()` ובדיקת מעגליות

```
function isFullAccount() {
  return isSignedIn()
    && request.auth.token.firebase.sign_in_provider !== 'anonymous';
}
```

בדיקת המעגליות — נקייה. הפונקציה אינה קוראת שום מסמך: אין בה `get()` ואין `exists()`. לכן אין תלות הרשאות מעגלית, וזה לא מקרי — מימוש חלופי מתבקש כמו "בדוק ב- `/users/{uid}` אם `isGuest` שקרי" היה יוצר בדיוק את המעגל: השער על קריאת המסמך היה זקוק לקריאת אותו מסמך. הבדיקה הטוקנית חסינה לזה מעצם הגדרתה.

נכונות הטענה שהבלוק תמיד קיים. ההערה בקוד מתעדת מדידה, לא הנחה: `token.firebase` נוכח בכל `emulator` — ID token ופרודקשן — ולכן הביטוי לא זורק. נמדד ל- `google.com`, `apple.com`, `password`, `anonymous` ו- `custom`. מכאן גם נובע שכל 249 בדיקות ה-`rules` האחרות, שמשתמשות ב- `authenticatedContext(uid)` ומקבלות `provider` 'custom', נקראות כחשבון מלא וממשיכות לעבור בלי שינוי.

למה לא `get('firebase', {})`. הניסוח הבטוח יותר לא נבחר כי `/games` כבר יושב על תקרת 1000 הביטויים. זו החלטה מתועדת ומודעת, והיא נכונה בהינתן שהבלוק נמדד כקיים.

הפער היחיד שנשאר. `sign_in_provider` מתאר את **אירוע ההתחברות**, לא את ספקי החשבון. §4 הוכיח על ה-`emulator` שאחרי `linkWithCredential` האירוע מדווח כספק החדש ולא כ-`anonymous`. זו הנקודה שראוי לאמת פעם אחת על מכשיר אמיתי — היא התלות היחידה במודל שאיננו שולטים בה.

בלי קוד תקיפה ובלי scraping — תיאור מעמדות והשגים בלבד.

התוקף	מה נדרש ממנו	מה הוא משיג היום	אחרי C או A
אורח אנונימי — כל מתקין	פתיחת האפליקציה. אין אימות, אין אימייל, אין מספר.	אינדקס מלא של בסיס המשתמשים: שם, אימייל, טוקני push, קואורדינטות אזור בית, זמינות, שיוך מזמין, העדפות, סטטיסטיקה. כולל query לפי אימייל וכולל query לפי מזמין.	המסמך של עצמו, וארבעה שדות ציבוריים לכל אחד אחר לפי uid.
חשבון מלא זדוני	הרשמה אמיתית.	אותו דבר בדיוק.	אותו דבר בדיוק. ראו את ההערה מתחת.
לא מחובר	—	כלום מ־ /users . ה־ /usersPublic אינו קריא אפילו לו.	ארבעה שדות לפי uid, למען דפי ה־web.
לקוח שעבר שינוי	בינארי מתוקן.	לכתוב שם תצוגה חופשי (§9) ולפרסם אותו דרך המראה.	זהה — הבדיקה בלקוח, לא ב־rules.

מה המודל החדש אינו מתיימר לפתור

חשבון מלא ממשיך למנות את /users אחרי המיגרציה. זה מודע: ההערה ב־rules מנסחת אותו במפורש — "חשבון אמיתי יכול כבר למנות את /users בעצמו, וארבעה שדות ציבוריים הם פחות מזה". כלומר isFullAccount() מעלה את מחיר הכניסה מאפס להרשמה מאומתת ולזיהוי; הוא לא הופך את /users לפרטי. אם המטרה היא לסגור את המניה לגמרי, זה ריפקטור אחר — העברת כל הצרכנים ל־projections — ואין לו מקום בסבב הזה. חשוב לא לתאר את המיגרציה כמשהו שהיא לא.

Obscurity אינה הגנה, ולא נחשבה כאחת. הדליפה הנוכחית אינה מוגנת בכך ש"אף אחד לא יודע את שמות האוספים" או ש"צריך לדעת §4 uid: הוכיח ש־list פתוח, כלומר גם הידיעה הזו אינה נדרשת. וגם אם הייתה — שמות האוספים נמצאים ב־JS של הלקוח.

מזהה	חומרה	הממצא
R-B-2	P0	/users קריא ל־get וגם list לכל session מאומת, אנונימי כולל. חשיפה של אימייל, טוקני push וקואורדינטות אזור בית לכל מי שמתקין. הועלה בסבב הזה מ"קריאה לפי uid" ל"אינדקס מוריד", על בסיס ההוכחה ב־§4.
R-B-3	P1	ל־ /usersPublic אין match בפרודקשן ואין catch-all, ולכן המראה נדחית לכולם. כל משטח שנבנה עליה ריק בשקט. חוסם מוצרי, לא אבטחתי.
R-C-1	P2	אין טלמטריית אימוץ גרסאות; ההחלטה על תזמון deploy תלויה במקור חיצוני (Play/ASC). ראו §6.
R-C-2	P2	describeThisDevice() — פונקציה מתה שבונה appVersion ואינה נקראת. הייתה יכולה להיות מקור הטלמטריה החסר.
R-C-3	P2	אין בדיקת rules אחת ל־קריאות של בינארי ישן. oldClientCompat מכסה כתיבות בלבד. ראו §19.
R-D-1	נמוך	name ללא אימות בשרת, מפורסם דרך המראה. ראו §9.

מזהה	חומרה	הממצא
R-D-2	נמוך	הידרדרות אורח בבינארי ישן שקטה לחלוטין — אין מדידה של הדחיות האלה אחרי deploy, כי אורח אינו יכול לכתוב ל- errors.

18 App Check — איכפה בפועל, לפי שירות

שירות	איכפה בפועל	העדות
Cloud Functions — callables	כבויה לחלוטין	const ENFORCE_APP_CHECK = false ב- functions/src/index.ts:136. 37 מופעים של enforceAppCheck: 36 דרך הקבוע, אחד false מקובע. אפס מופעי true. כלומר אף callable אחד אינו אוקף.
Firestore	לא נקבע מהריפו	אכיפת App Check על Firestore היא הגדרת קונסולה, לא קוד. לא ניתן לקבוע אותה מהמאגר, ולא נשאלה בסבב הזה כדי לא לגעת בפרודקשן.
Storage	לא נקבע מהריפו	כנ"ל.
הלקוח	מאתחל	src/firebase/appCheck.ts מאתחל דרך ה-provider הנייטיב, ונופל בחן אם המודול לא מקושר.

המשמעות למודל האיומים: הלקוח מייצר טוקני App Check, והשרת אינו דורש אותם באף נקודה שמשלטת מהקוד. לכן **אין שכבת App Check שמרככת את R-B-2** — צד שאינו האפליקציה יכול לדבר עם Firestore ישירות, וההגנה היחידה היא ה-rules. ההערה בקוד מתעדת את הסיבה (חסימת iOS לפני שה-App Attest אומת) ואת תנאי ההחזרה. זה נותר חוב פתוח ואינו חלק מהמיגרציה.

19 מטריצת בדיקות ה-rules — כיסוי ופערים

15 קבצי בדיקות, בסיס של 268 בדיקות. לא נוספה ולא שונתה שום בדיקה בסבב הזה.

קובץ	מקרים	מה הוא מוכיח
anonymousAccess	19	לב המיגרציה: טענת הטוקן לכל ספק, מה אורח יכול ולא יכול, המראה כ-world-readable וכ-client-unwritable, הרוסטר לשני סוגי הצופים, ושימור מלא של כל קריאה של חשבון מלא. כולל list נפרד מ-get.
firestore	38	הבסיס הרחב.
gameReadDeleted	13	קריאת משחק שנמחק.
/ antiHijack clubIsolation	10 / 10	חטיפת רוסטר, בידוד בין מועדונים.
/ publishTeams roundHistoryAccess	7 / 7	פרסום כוחות, גישה להיסטוריה.
oldClientCompat	3	כתיבות של בינארי ישן בלבד — ביטול הרשמה במודל ההצעה ובמקרה הפשוט.
שאר הקבצים	0	test(, joinRequest, gameListRegression, displayName, availabilitySave, seasonSummary, seasonCloseWindow, notificationSender — מבנים אחרים או תרחישי הרצה ידניים.

הפערים, מסומנים ולא נסגרים

- אין בדיקת קריאה לבינארי ישן. oldClientCompat בודק כתיבות. הטבלה ב-§5 נבנתה בסבב הזה על probe חד-פעמי בתיקיית scratchpad, לא על בדיקה במאגר. זה הפער המשמעותי — הוא בדיוק מה ש-deploy צריך לאמת.
 - אין בדיקה לטוקן שאחרי link. rules-unit-testing מטביע טוקנים ישירות, ולכן הוא לא יכול לבדוק את הזרימה האמיתית של linkWithCredential. הפער אמיתי ולא נסגר ע"י בדיקת rules — הוא דורש אימות על מכשיר.
 - אין בדיקה שמסמך משתמש חי ללא מראה נראה כשם ריק. זו התלות ב-§8, ואין לה ביטוי בבדיקות.
 - שלוש בדיקות נכשלות בבסיס — כשל קיים מלפני הסבב, לא נגרם ולא טופל כאן.
- הערת שיטה ששווה לשמור: את הבדיקות האלה מריצים סדרתית (npm run test:rules , --test-concurrency=1), ואסור לבדוד חוק בקובץ מוקטן — ה-catch-all עונה ראשון והתוצאה מטעה.

20 סדר הפריסה המומלץ

ההמלצה נשענת על הפרדה אחת: B הוא הרחבה טהורה, C הוא תיקון אבטחה, A מוסיף עליהם רק נוחות מוצרית. לכן הסדר אינו "הכול או כלום".

שלב	הפעולה	למה כאן, ולא לפני או אחרי
0	הרצת auditUsersPublic.mjs על פרודקשן — קריאה בלבד, דורשת אישור מפורש	כל עוד /users פתוח, מראה חסרה נסתרת. אחרי סגירה היא הופכת לשם ריק שאף אחד לא ירשום. זה תנאי מקדים, ולכן הוא שלב 0 ולא שלב אחרון.

שלב	הפעולה	למה כאן, ולא לפני או אחרי
1	אימות <code>sign_in_provider</code> <code>linkWithCredential</code> על מכשיר אמיתי	התלות היחידה במודל שאינה בשליטתנו (§15). בדיקה אחת, לפני שכל השאר נשען עליה.
2	אפשרות B — בלוק <code>/usersPublic</code> בלבד	מסיר את R-B-3 בלי לשבור דבר, בהוכחה: האוסף הוא default-deny היום ואף לקוח לא תלוי בהתנהגותו. זה גם מה שמפעיל את המראה כך שסגירת <code>/users</code> בשלב 3 תהיה כבר מגובה בנפילה חנינית ללקוח החדש.
3	אפשרות C — שלוש השורות על <code>/users</code>	מסיר את R-B-2 — כל ה-PII — במחיר של שני משטחים לאורחים על בינארי ישן. אחרי שלב 2, לקוח חדש כבר קורא מהמראה, ולכן העלות מתכנסת לבינאריים ישנים בלבד.
4	שחרור הגרסה עם הריפקטור	מכאן ואילך אורחים חדשים מקבלים את החוויה המלאה מהמראה.
5	שמונה נקודות <code>isFullAccount()</code> הנותרות — <code>rounds</code> , <code>playerStats</code> , <code>pairStats</code> , <code>ratings</code> , <code>seasonTitles</code>	אלה אינן PII ואינן דחופות. יש להן אפס תועלת אבטחתית שלא הושגה בשלב 3, והן כן שוברות משטחים לאורחים על בינארי ישן. הן צריכות לחכות לאימוץ — כאן ההמתנה של אפשרות D היא הנכונה, ורק כאן.

הפיצול הזה הוא עיקר התשובה של הסבב: אפשרות D אינה שגויה כשיטה — היא פשוט יושמה על הדבר הלא נכון. נכון לחכות עם שלב 5. לא נכון לחכות עם שלב 3.

21 האם R-B-2 חמור דיו להצדיק שינוי rules לפני שהריפקטור והגרסה הסתיימו?

דחיפות אבטחתית — מופרדת מנוחות מוצרית

- כן, והתשובה התחדדה בסבב הזה. לפני ההוכחה ב-§4 היה אפשר לתאר את R-B-2 כ"כל מי שמחובר יכול לקרוא מסמך משתמש אם הוא יודע את ה-uid" — חשיפה שדורשת ידיעה מוקדמת ומניבה מסמך אחד. אחרי ההוכחה זה תיאור שגוי. מה שקיים בפועל:
- עלות הכניסה היא אפס: התקנה אחת נותנת session אנונימי, בלי אימייל, בלי מספר טלפון, בלי זיהוי.
 - ההשג הוא האוסף כולו: `list` מותר, ולכן אין צורך לדעת אף uid.
 - המידע כולל אימייל וקואורדינטות אזור הבית של אנשים אמיתיים — צירוף שהוא נתון אישי בכל קריאה סבירה של הדין, ולא "מטא-דאטה".
 - הוא כולל טוקני `push`, כלומר ערוץ התראות ישיר למכשירים.
 - אין שכבה מרכזת: App Check אינו אוסף באף (§18) callable, ואין obscurity שנחשבת כאן כהגנה (§16).

נוחות מוצרית — מה שאינו דחוף

מה שאינו מצדיק deploy מוקדם: הפעלת המראה כדי שהרוסטר של האורח ירנדר, וסגירת `rounds` / `playerStats` / `ratings` / `pairStats` / `seasonTitles`. הראשון הוא חוסם מוצרי שמחירו "המסך נראה רע", והוא נפתר גם בשלב 2 חסר-הסיכון. השני הוא ניקיון מודל בלי רווח אבטחתי מעל שלב 3. שניהם יכולים לחכות לאימוץ. הראשון לא צריך.

הפסק

הדחיפות היא של

סגירת `/users`

— ולא של הריפקטור, לא של המראה, ולא של שמונה הנקודות הנותרות. לכן התשובה היא לא "לפרוס את ה-ruleset החדש עכשיו" אלא "לפרוס עכשיו את החלק שסוגר את הדליפה, ולהשאיר את השאר בלוח הזמנים של הגרסה". זה בדיוק שלבים 2–3 של §20, וזה בדיוק §22.

שני עריכות ב־ `firestore.rules`. לא יותר. אין לגעת בשמונה נקודות ה־ `isFullAccount()` הנותרות, אין לגעת ב־ `Functions`, אין לגעת ב־ `Hosting`, ואין לשנות `minimumSupportedVersion`.

עריכה 1 — הוספת האוסף (הרחבה טהורה, אפס סיכון)

```
match /usersPublic/{uid} {
  allow get: if true;
  allow list: if isFullAccount();
  allow write: if false;
}
```

עריכה 2 — החלפת השורה האחת שמדליפה

```
// במקום, ב־
match /users/{uid}:
- allow read: if isSignedIn();
+ allow get: if isSelf(uid) || isFullAccount();
+ allow list: if isFullAccount();
```

`create` על `/users` — להשאיר כפי שהוא בפרודקשן. הוספת `isFullAccount()` שם אינה מוסיפה הגנה על נתונים (אדם יכול ליצור רק את המסמך של עצמו בשני המצבים), והיא כן מוסיפה את התלות היחידה שלא אומתה על מכשיר אמיתי (§15). ההפרדה הזו היא מה שהופך את השינוי הזה ל"הקטן ביותר ובטוח".

מה השינוי הזה מבטיח, ומה לא

מה	אחרי השינוי
אורח אנונימי קורא <code>/users</code> של נחם — אימייל, טוקנים, קוארדינטות אחר	
אורח אנונימי ממנה את <code>/users</code> נחם	
אורח קורא את המסמך של עצמו	נשמר, דרך <code>isSelf(uid)</code>
אורח יוצר את המסמך של עצמו	נשמר — <code>create</code> לא נוגעים בו
חשבון מלא — כל קריאה שיש לו היום	נשמר במלואו, כולל <code>documentId() in</code>
המראה הופכת לקריאה	כן — סוגר את R-B-3 גם
אורח על בינארי ישן	מידרדר בשקט שמות רוסטר ל־ "...", כרטיס שחקן "שחקן לא נמצא". ללא קריסה — <code>try/catch</code> קיים בשני המסלולים, נבדק ב־ <code>git</code> .
אורח על הבינארי החדש	חוויה מלאה מהמראה

מה חייב לקרות לפני הרצת ה־ `deploy` הזה

- `npm run test:rules` **סדרתית** (`JAVA_HOME=/opt/homebrew/opt/openjdk`) — ולוודא שמספר הכשלים נשאר שלוש, ולא ארבע.
- הרצת `auditUsersPublic.mjs` על פרודקשן (קריאה בלבד, דורשת אישור) — `missing` ו־ `blankName` הם מה שהופך לנראה ברגע הסגירה.
- אימות אחד על מכשיר: אורח → הרשמה → קריאת מסמך המשתמש שלו.

4. תוכנית גלגול לאחר: ה-ruleset בפרודקשן הוא 867bea46-48ed-45c9-9a3b-57d693443bbf . הוא עצמו הגלגול לאחר של תקרית is list ב-22.09, ולכן הוא מוכח-עובד — וזה מה שהופך את החזרה אליו לזולה.

מפורשות

הנוסח לעיל הוא ניסוח להחלטה, לא פעולה. לא נערך אף קובץ, לא נעשה commit, ולא נפרס דבר. **ההמלצה לא יושמה.**

23 שיטה, גבולות ומה לא נעשה

מה נעשה

- diff מונטי על נתיבי match מנורמלים — לא השוואת טקסט.
- שלוש הרצות probe על emulator מקומי מול demo-soccer : get מול list על שני ה-rulesets, קריאה/כתיבה של עצמי, ומטריצת הקריאות של הבינארי הישן. אותה תשתית שמריצה את 268 בדיקות ה-rules.
- הרצת probe אחת על Auth emulator מול demo-soccer לשאלת ה-sign_in_provider אחרי link — פעמיים, עם רענון מאולץ ובלעדיו.
- קריאת קוד: firestore.rules , functions/src/index.ts , groupService , userService , authUpgrade , joryio , notificationsService , firebase/firestore.ts , appCheck.ts , ומסכי הקהילה והשחקן.
- ארכיטקטורה ב-git: תיארוך כניסת מצב האורח והמראה, והתנהגות הטיפול בשגיאות בגרסאות הקודמות של שני מסלולי ההידרציה.

מה לא נעשה, במכוון

- אפס קריאות לפרודקשן. אין probing, אין mass reads, אין כתיבה, אין יצירת חשבון, אין שינוי נתונים.
- לא הורץ auditUsersPublic.mjs — הוא סורק בעמודים של 300 מסמכים, וזה mass read.
- לא נשאל Play / ASC / Joryio לנתוני אימוץ. נאמר במקום זה מה שיש ומה שאין (§6).
- לא נערך firestore.rules , לא קוד, לא Functions, לא Hosting, לא minimumSupportedVersion .
- לא נוספו ולא שונו בדיקות. פערי הכיסוי סומנו ב-§19 ונשארו פתוחים.
- אין commit ואין deploy. git status ריק, HEAD נשאר 4fbf441 .
- ללא Web ו-iOS — מחוץ להיקף הסבב.
- אין קוד תקיפה ואין scraping בדוח. §16 מתאר מעמדות והשגים בלבד.
- קובצי ה-probe נכתבו בתיקיית ה-scratchpad מחוץ למאגר, ולכן לא נגעו בעץ העבודה.

מה שהדוח הזה לא יכול להבטיח

- התנהגות ה-provider אחרי link בפרודקשן. נמדד על emulator. זו התלות היחידה במודל שאינה בשליטת הקוד שלנו, והיא דורשת אימות אחד על מכשיר (§15, שלב 1 ב-§20).
- שלמות המראה בפועל. הידע הוא מ-24.09 ולא אומת מחדש (§8).
- אכיפת App Check על Firestore ו-Storage. הגדרת קונסולה שלא ניתן לקבוע מהמאגר (§18).
- תפוצת הבינאריים. אין טלמטריה פנימית. כל החלטת תזמון חייבת להישען על Play Console ו-ASC (§6).